

Take-Home Case Study: (Senior) Data & AI Consultant

Ziel & Kontext

Bei Hitachi Solutions entwickeln wir intelligente Lösungen im Microsoft-Ökosystem, die Unternehmen helfen, unstrukturierte Informationen in verwertbare Daten zu verwandeln. Einer der häufigsten Anwendungsfälle in der Praxis: **Dokumente wie Rechnungen, Verträge, Berichte oder Formulare** enthalten wertvolle Informationen, die heute noch manuell abgetippt oder kopiert werden.

Mit dieser Case Study möchten wir sehen, wie du ein **reales AI-Problem End-to-End löst** – von der Problemdefinition über die KI-gestützte Datenextraktion bis hin zur strukturierten Ablage.

Aufgabenstellung

Entwickle eine KI-gestützte Pipeline, die strukturierte Daten aus unstrukturierten Dokumenten (PDFs, Bilder o. ä.) extrahiert, transformiert und in ein strukturiertes Zielformat überführt (Datenbank, strukturiertes File oder API).

Du hast die **Freiheit, den konkreten Dokumententyp und Use Case selbst zu wählen**. Beispiele (nicht abschließend):

Use Case	Dokument-Typ	Extrahierte Daten
 Rechnungsverarbeitung	PDF-Rechnungen (z. B. selbst generierte Beispielrechnungen)	Rechnungsnummer, Datum, Positionen, Beträge, USt., Lieferant
 Lebenslauf-Parsing	PDF-Lebensläufe (z. B. von öffentlichen Templates)	Name, Skills, Berufserfahrung, Ausbildung, Kontaktdaten
 Immobilien-Exposés	PDF-Exposés (z. B. von Immoscout-Beispielen)	Fläche, Preis, Lage, Ausstattung, Baujahr
 Geschäftsberichte	Annual Reports (z. B. öffentliche Unternehmensberichte)	KPIs, Umsatz, Gewinn, Mitarbeiterzahlen, Segmentdaten
 Medizinische Befunde	Beispiel-Laborbefunde (synthetisch generiert)	Patientendaten, Messwerte, Referenzbereiche, Diagnosen

Tipp: Erstelle dir 5–10 Beispieldokumente selbst (oder nutze öffentlich verfügbare Samples). Das zeigt uns auch, wie du mit Testdaten umgehst.

✅ Pflicht-Anforderungen (Must-Haves)

1. Use Case Dokumentation 📄

- Beschreibe **klar und prägnant**, welches Problem du löst und für wen (Zielgruppe/Persona)
- Definiere die **Input-Dokumente** und das **gewünschte Output-Schema**
- Erkläre den **Business-Mehrwert**: Was wird heute manuell gemacht? Was spart die Lösung?
- Definiere **Erfolgsmetriken** (z. B. Extraktionsgenauigkeit, verarbeitete Dokumente pro Minute)

2. Architektur & Design 🏗️

- Erstelle ein **Architekturdiagramm** (draw.io, Mermaid o. ä.) der gesamten Pipeline:

Dokument-Input → Preprocessing → AI-Extraktion → Transformation/Validation → Strukturierte Ablage

- Begründe deine **Technologieentscheidungen** und diskutiere Alternativen/Trade-offs
- Zeige ein **Azure-Mapping**: „Lokal nutze ich X – in Produktion würde ich Azure Service Y einsetzen" (z. B. Azure Document Intelligence, Azure Blob Storage, Azure SQL, Cosmos DB)
- Beschreibe, wie die Lösung mit **verschiedenen Dokumentenlayouts** oder **Varianten** umgeht

3. AI-gestützte Extraktion (Python – Pflicht) 🤖

Wähle einen oder mehrere Ansätze für die Structured Data Extraction:

Ansatz	Beispiel-Tools (alle kostenlos nutzbar)
LLM-basiert	OpenAI API (Free Credits), Ollama + lokale Modelle (Llama, Mistral), LangChain
OCR + NLP	Tesseract OCR, PyMuPDF, pdfplumber + spaCy / regex-basierte Extraktion
Document AI	Azure Document Intelligence (Free Tier: 500 Seiten/Monat), Unstructured.io

Hybrid	Kombination aus OCR + LLM-Nachverarbeitung
--------	--

Erwartungen an die Implementierung:

- Funktionsfähiger **Python-Code**, der aus Dokumenten strukturierte Daten extrahiert
- Ein klar definiertes **Output-Schema** (z. B. Pydantic-Modelle, JSON Schema, Dataclass)
- **Umgang mit Varianz**: Die Pipeline sollte mit leicht unterschiedlichen Dokumentenformaten zurechtkommen
- **Confidence Scores** oder Validierungslogik: Wie sicher ist die Extraktion? Was passiert bei unsicheren Ergebnissen?

4. Transformation & Ablage

- **Transformation**: Cleaning, Normalisierung, Enrichment der extrahierten Daten (z. B. Datumsformate vereinheitlichen, Beträge parsen, Kategorien zuordnen)
- **Strukturierte Ablage** in mindestens einem Ziel:
 - Lokale Datenbank (SQLite, DuckDB, PostgreSQL via Docker)
 - Strukturiertes File (CSV, Parquet, JSON)
 - Optional: SharePoint-Liste oder REST-API-Endpunkt
- **Schema-Design**: Durchdachtes Datenbankschema / Datenmodell mit Begründung

5. Codequalität & Best Practices

- Saubere **Projektstruktur** (z. B. src/, tests/, docs/, config/, sample_data/)
 - Nachvollziehbare **Git-Commit-Historie**
 - **Error Handling** und Logging (Was passiert, wenn ein PDF nicht lesbar ist?)
 - **Konfigurationsmanagement** (keine hardcodierten API-Keys, .env-basiert)
 - **Type Hints** und Docstrings
-

★ Bonus-Punkte (Nice-to-Haves)

Bereich	Beschreibung
C# / .NET	Ein Microservice in C# (z. B. ein REST-API-Endpunkt, der Dokumente entgegennimmt und die Extraktion triggert, oder ein Validation-Service)
Docker	Containerisierung der Lösung (docker-compose up zum Starten)
Prompt Engineering	Falls LLM-basiert: Dokumentiertes Prompt Engineering mit verschiedenen Prompt-Varianten und deren Ergebnisvergleich
Evaluation	Systematische Bewertung der Extraktionsqualität (z. B. Ground-Truth-Vergleich, Accuracy pro Feld)
Data Quality	Validierungsregeln für die extrahierten Daten (Plausibilitätschecks, Pflichtfelder, Wertebereiche)
UI / Demo	Einfaches Frontend (z. B. Streamlit), über das man ein Dokument hochladen und das Ergebnis sehen kann
Batch-Verarbeitung	Verarbeitung mehrerer Dokumente in einem Durchlauf mit Fortschrittsanzeige und Fehlerprotokoll

🔧 Erlaubte Tools & Technologien

Alle Tools müssen **kostenlos oder mit Free Tier** nutzbar sein:

- **Sprache:** Python (Pflicht), C# / .NET (optional)
- **AI/ML:** Ollama (lokal, kostenlos), OpenAI API (Free Credits), Azure Document Intelligence (Free Tier), Tesseract, spaCy, LangChain, Hugging Face
- **Datenbank:** SQLite, DuckDB, PostgreSQL (via Docker)
- **CI/CD:** GitHub Actions (Free Tier)
- **Containerisierung:** Docker / Docker Compose
- **Versionierung:** Git / GitHub
- **PDF-Processing:** PyMuPDF, pdfplumber, pdf2image, Camelot

⚠️ **Es sind keine kostenpflichtigen Lizenzen oder Cloud-Abonnements erforderlich.** Alles muss lokal oder mit Free-Tier-Diensten umsetzbar sein.